MEMORY-LAYOUT-BANDWIDTH

엔디언·메모리 인터리빙·캐시 일관성

연관 기출 · 126-1-11, 128-1-11, 137-2-1

한눈에 이해

바이트를 어떤 순서로 놓을지, 연속 접근을 여러 메모리에 어떻게 나눌지, 복제된 캐시 값의 충돌을 어떻게 막을지 구분합니다.

정의와 핵심 원리

엔디언은 여러 바이트로 이루어진 값을 메모리에 놓는 순서입니다. 메모리 인터리빙은 주소를 여러 뱅크·채널에 나눠 병렬 접근을 돕는 배치 기법입니다. 캐시가 데이터를 복제하면 코어 사이에서 사본의 일관성도 관리해야 합니다.

• 빅엔디언은 최상위 바이트를, 리틀엔디언은 최하위 바이트를 낮은 주소에 둡니다. 값 자체나 비트의 중요도는 그대로입니다. 통신이나 파일 형식에서 지정한 바이트 순서와 호스트의 순서가 다르면 직렬화할 때 변환합니다.

• 저위 인터리빙은 주소의 하위 비트로 뱅크를 고릅니다. 연속 주소가 다른 뱅크에 배정돼 병렬 접근을 돕습니다. 일정한 간격으로 접근하면 같은 뱅크에 몰려 충돌할 수 있습니다. 캐시 라인·채널·NUMA의 배치는 장치 구성을 확인합니다.

• write-through는 쓰기를 하위 계층에 바로 반영합니다. write-back은 캐시 라인을 변경된 상태(dirty)로 두었다가 캐시에서 내보낼 때 등에 반영합니다. 여러 코어의 사본이 낡는 문제는 스누핑이나 디렉터리 기반의 MESI 계열 프로토콜 등으로 소유권과 무효화를 관리합니다. 캐시 일관성과 메모리 일관성 모델은 구분합니다.

구조·비교·판단

구분	핵심 원리	적용·한계
엔디언	값을 구성하는 바이트의 순서	형식 명세와 변환
인터리빙	주소를 여러 뱅크에 배치	대역폭 증가와 뱅크 충돌
캐시 일관성	여러 사본에 쓰기 반영	거짓 공유(false sharing)와 프로토콜 비용

적용 예제

바이트와 뱅크 배치

강사 생성 학습 예제입니다. 공식 기준답안이 아닙니다.

조건: 바이트 순서 예제는 32비트 값 0x12345678을 바이트 주소 100~103에 둡니다. 뱅크 배치 예제는 별도로 워드 주소를 사용하며 뱅크는 4개입니다.

1. 낮은 주소부터 빅엔디언은 12, 34, 56, 78이고, 리틀엔디언은 78, 56, 34, 12입니다. 각 바이트는 16진수로 표기했습니다.

2. 워드 주소를 4로 나눈 나머지가 뱅크 번호라면 워드 주소 0, 1, 2, 3은 서로 다른 뱅크에 들어갑니다.

3. 워드 주소 0, 4, 8, 12는 모두 뱅크 0에 들어갑니다. 연속 주소와 같은 병렬성을 기대하기 어렵습니다.

4. 두 코어가 같은 캐시 라인에 있는 서로 다른 변수를 수정해도 거짓 공유가 발생할 수 있습니다.

결과·해석: 바이트의 저장 순서, 주소의 뱅크 배치, 캐시 사본 관리는 서로 다른 문제입니다.

전제·주의: 엔디언·인터리빙·MESI 상세의 직접 명세는 추가 확인이 필요하며 DMA 공식 문서로 주소·일관성 기본 범위만 검증했습니다.

답안 골격과 혼동 방지

계층 구조 → 엔디언 예제 → 뱅크 번호 계산 → 쓰기 정책과 캐시 일관성 → 성능의 장단점

- write-through가 동시성 잠금을 대체하지 않습니다.
- 주소는 낮은 곳부터 높은 곳으로 읽고, 엔디언에 따라 각 주소의 바이트가 달라집니다.

내용을 가리고 회상

1. 0x12345678을 리틀엔디언으로 저장할 때 가장 낮은 주소의 바이트는 무엇입니까?

2. 뱅크가 4개인 예에서 어떤 주소 간격으로 접근하면 충돌합니까?

3. 캐시 일관성과 프로그램 동기화는 어떻게 다릅니까?

적용 기준과 근거

명시한 출처 판본의 원리를 설명하며 제품별 구현 차이는 별도 확인합니다.

내용 검토 상태: 부분 확인. 출처 확인은 공식 정답이나 학습 성취를 뜻하지 않습니다.

출처별 상세 열람 범위는 위키의 출처 기록에서 확인합니다.

- Dynamic DMA mapping Guide - Linux Kernel documentation; 상시 문서; 확인 2026-11-05.